

Advanced Process Mining — Assignment Part II

Group $\langle \text{GroupID} \rangle$
 $\langle \text{Member 1} \rangle, \langle \text{Member 2} \rangle, \langle \text{Member 3} \rangle$

July 5, 2026

Contents

1	Question 3: Trace Probabilities in GSLPNs	2
2	Question 5: Object-Centric Process Mining	2
2.1	(b) Conformance checking for Cameron	5

1 Question 3: Trace Probabilities in GSLPNs

(a) Stochastic path with highest probability

Step	Trace so far	Marking	Transition fired	$P(\text{step})$
0	$\langle \rangle$	$M_0 = \{p_0\}$	t_0	1
1	$\langle a \rangle$	$M_1 = \{p_1, p_2\}$	t_2	$\frac{3}{5}$
2	$\langle a, d \rangle$	$M_2 = \{\}$	t_d	$\frac{1}{1}$
3	$\langle a, d, c \rangle$	$M_3 = \{\}$	t_c	$\frac{1}{1}$
4	$\langle a, d, c, e \rangle$	$M_4 = \{\}$	t_e	$\frac{1}{1}$
5	$\langle a, d, c, e, f \rangle$	$M_5 = \{\}$	t_f	$\frac{1}{1}$

Table 1: Path π_1 : $\langle a, d, c, e, f \rangle$ via $(t_0, t_2, t_3, t_5, t_{10}, t_{11}, t_6, t_4, t_6, t_7)$

$$P(\pi_1) = [\text{TODO: prob}] \cdot [\text{TODO: prob}] \cdot [\text{TODO: prob}] \cdot [\text{TODO: prob}] \cdot [\text{TODO: prob}] = [\text{TODO: value}]$$

Step	Trace so far	Marking	Transition fired	$P(\text{step})$
0	$\langle \rangle$	$M_0 = [\text{TODO: marking}]$	–	–
1	$\langle a \rangle$	$M_1 = [\text{TODO: marking}]$ [TODO: ...]	t_a	[TODO: prob]
5	$\langle a, d, c, e, f \rangle$	$M_5 = [\text{TODO: marking}]$	t_f	[TODO: prob]

Table 2: Path π_2 : $\langle a, d, c, e, f \rangle$ via [TODO: describe alternate routing]

$$P(\pi_2) = [\text{TODO: value}]$$

$$\max(P(\pi_1), P(\pi_2), \dots) = P([\text{TODO: } \pi_i]) = [\text{TODO: value}]$$

[TODO: State which path attains the maximum and give a one-sentence justification.]

(b) Probability of observing trace σ

$$\begin{aligned}
 P(\sigma) &= P(\pi_1) + P(\pi_2) + \dots \\
 &= [\text{TODO: value}] + [\text{TODO: value}] + \dots \\
 &= [\text{TODO: final value}]
 \end{aligned}$$

[TODO: Briefly justify why exactly these paths (from part (a)) are the ones that project onto σ , and confirm no other path was missed.]

2 Question 5: Object-Centric Process Mining

The object-centric event log L records a collaborative coding process in a Git-based version control system. The object types are $OT = \{\text{branches, users, files, file versions}\}$ and the event types are $ET = \{\text{commit, pull, merge, push}\}$. File versions are linked to their parent file via the object-object relation `is_a`. When users commit, event-object qualifiers `produce` (new version) and `consume` (old version) record which file versions were created or superseded. The full log is provided as an OCEL 2.0 file `version_control.sqlite`.

(a) Checking C1, C2, C3 with OCPQ (4.5 points)

For each constraint we describe the OCPQ query used to check it against the event log L , show a screenshot of the query and its result, and briefly explain the underlying logic.

C1: “You should pull before you push, and you should not do anything between the pull and the push except for possibly a merge.”

Query. We use a nested query tree with three levels.

- **Root node** – all push events on a branch: variables e_2 : push, o_1 : branches; filter $E2O(e_2, o_1)$.
- **Child node** – candidate pulls preceding this push on the same branch: variable e_1 : pull (inherits o_1); filters $E2O(e_1, o_1)$ and $e1.time() < e2.time()$.
- **Grandchild node** – events strictly between e_1 and e_2 , excluding merges: variable e_3 : *; filters $e1.id() \neq e3.id() \ \&\& \ e2.id() \neq e3.id(), e1.time() < e3.time(), e3.time() < e2.time(), e3.attr("ocel:activity") \neq "merge"$.

Constraints: on the *child* (pull) node we require $|grandchild| \leq 0$, which keeps only the *clean* pull witnesses – pulls with nothing (other than possibly a merge) between them and the push. On the *root* (push) node we require $|child| \geq 1$: a push is compliant iff at least one clean witness pull remains.

Result and explanation. The query flags 4 of the 30 pushes as violations (see Figure 1). A push is only counted as compliant if it has an *immediately* preceding pull on the same branch, with no event other than a merge in between; existence of some *earlier*, non-adjacent pull is not sufficient, since anything (including another push or commit) sitting between that pull and the push disqualifies it as a witness. The four flagged pushes either have no preceding pull on their branch at all, or every earlier pull is separated from them by an intervening non-merge event.

C2: “Do not alternate between branches: after committing to one branch b , you should not touch a different branch until having pushed to b .”

Query. Again a three-level nested query tree.

- **Root node** – all commits: variables e_1 : commit, o_1 : branches, o_2 : users; filters $E2O(e_1, o_1), E2O(e_1, o_2)$.
- **Child node** – candidate later pushes to the same branch by the same user: variable e_2 : push (inherits o_1, o_2); filters $E2O(e_2, o_1), E2O(e_2, o_2), e1.time() < e2.time()$.
- **Grandchild node** – events by the same user on a *different* branch, strictly between e_1 and this candidate push: variable e_3 : *, o_3 : branches; filters $E2O(e_3, o_2), E2O(e_3, o_3), o3.id() \neq o1.id(), e1.time() < e3.time(), e3.time() < e2.time()$.

Constraints: on the *child* (push) node we require $|grandchild| \leq 0$, keeping only pushes to b that are not preceded by activity of the same user on a different branch. On the *root* (commit) node we require $|child| \geq 1$: a commit is compliant iff at least one such clean push exists afterwards.

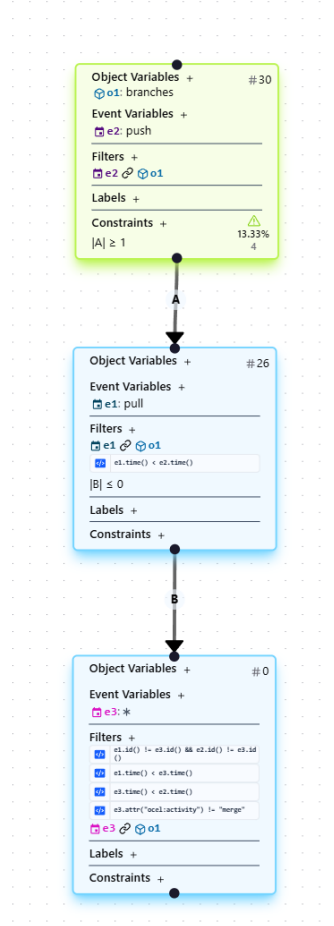


Figure 1: OCPQ query for C1 (three nested nodes) and result: 4 of 30 pushes violate C1 (86.67% compliance).

Result and explanation. 2 of the 39 commits violate C2 (see Figure 2). A commit is compliant if the committing user eventually pushes to the same branch again without first touching a different branch; committing again, or pulling, on the *same* branch b in between does not count against the constraint, only activity tied to a *different* branch object does. Of the two violating commits, at least one case corresponds to the user touching a different branch before ever pushing to b ; the other case may correspond to the user never pushing to b again at all – both are genuine violations of C2, but for different underlying reasons.

C3: “Avoid coordination problems: a coordination problem between two users $u_1 \neq u_2$ is a merge event that consumes two versions of the same file, where one version was committed by u_1 and the other by u_2 .”

Query. Unlike C1 and C2, C3 is not an existential (“does a witness exist”) condition but a direct pattern match, so a single flat query node suffices.

- Object variables: o_1, o_2 : file versions, o_3 : files, o_4, o_5 : users.
- Event variables: e_1 : merge, e_2, e_3 : commit.
- Filters: $E2O(e_1, o_1)@consume$, $E2O(e_1, o_2)@consume$, $O2O(o_1, o_3)@is_a$, $O2O(o_2, o_3)@is_a$, $E2O(e_2, o_1)@produce$, $E2O(e_3, o_2)@produce$, $E2O(e_2, o_4)$, $E2O(e_3, o_5)$; CEL constraints $o1.id() < o2.id()$ and $o4.id() != o5.id()$.

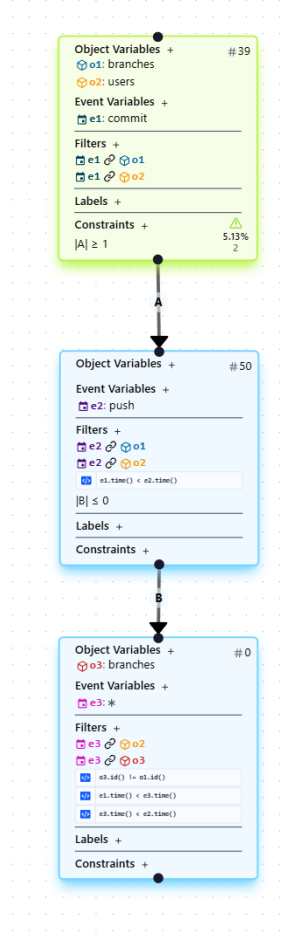


Figure 2: OCPQ query for C2 (three nested nodes) and result: 2 of 39 commits violate C2 (94.87% compliance).

The condition $o1.id() < o2.id()$ imposes a canonical ordering on the two conflicting versions so that each real coordination problem is counted exactly once (without it, every coordination problem would be found twice, as (o_1, o_2) and (o_2, o_1)). The condition $o4.id() \neq o5.id()$ is the actual C3 condition: the two versions were committed by different users.

Result and explanation. The query returns 6 output bindings (see Figure 3), i.e. 6 coordination problems: pairs of file versions of the same file, produced by two different users, that were both consumed by the same merge event. Each binding directly witnesses one instance of the pattern described by C3, so no separate compliance percentage is computed here – the count itself is the answer. Note that this counts *version pairs*; if a single merge consumed more than two conflicting versions it would contribute multiple pairs to this count rather than being counted once per merge.

2.1 (b) Conformance checking for Cameron

OCPN N_1

The accepting object-centric Petri net N_1 used for conformance checking is given in the assignment. Its transitions are **commit**, **pull**, **merge**, and **push**. The **branches** subnet and the **users** subnet each follow the pattern $\text{commit}^+ \rightarrow \text{pull} \rightarrow [\text{merge}]? \rightarrow \text{push}$ (with a loop for multiple rounds) where **merge** is optional.

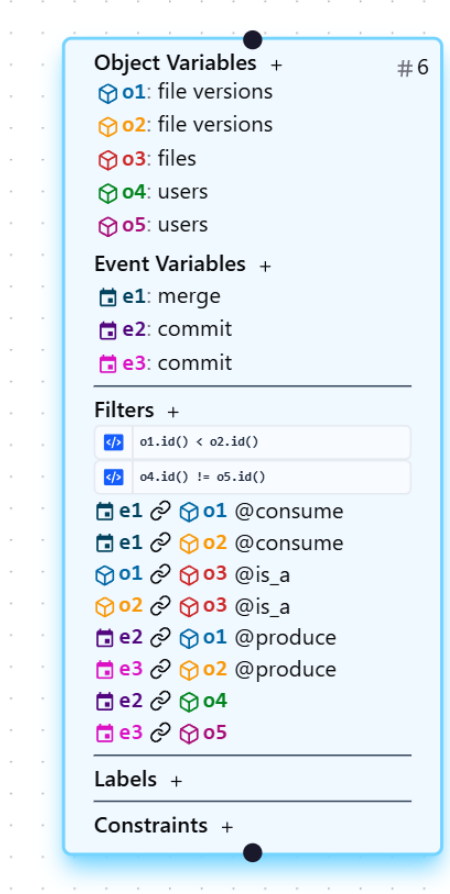


Figure 3: OCPQ query for C3 (single flat node) and result: 6 coordination problems found.

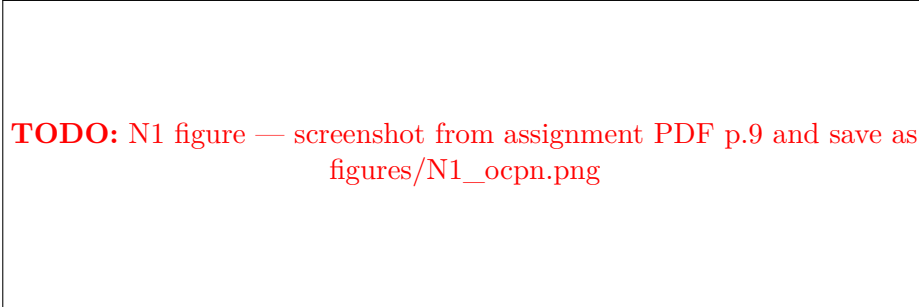


Figure 4: Accepting OCPN N_1 (from the assignment).

Constructing L_1^{Cameron}

L_1 is the view of L restricted to object types **branches** and **users** (i.e. **files** and **file versions** are removed). L_1^{Cameron} is obtained from L_1 by first removing all events in which *Cameron* is not involved, then removing all objects disconnected from the remaining events, and finally removing all dangling relations.

Flattened traces

Flattening L_1^{Cameron} to the **branches** object type yields one trace per branch that *Cameron* worked on; flattening to **users** yields a single trace for *Cameron*.

Table 3: Traces per branch (`branches` flattening of L_1^{Cameron}).

Branch	Trace
[TODO: paste branch traces from script output]	

Branches flattening.

Table 4: Trace for *Cameron* (`users` flattening of L_1^{Cameron}).

User	Trace
[TODO: paste user trace from script output]	

Users flattening.

Token-based replay fitness

Token-based replay is performed on each flattened log against the corresponding subnet of N_1 .

Table 5: Token replay diagnostics — `branches` subnet.

Trace (branch)	Fit?	Fitness	Consumed	Produced	Missing	Remaining
[TODO: paste from script]						
Average		[TODO: value]				

Branches subnet — token replay.

Table 6: Token replay diagnostics — `users` subnet.

Trace (user)	Fit?	Fitness	Consumed	Produced	Missing	Remaining
[TODO: paste from script]						
Average		[TODO: value]				

Users subnet — token replay.

Aggregated fitness. The average token-based replay fitness over both subnets is $\bar{f} = \frac{f_{\text{branches}} + f_{\text{users}}}{2} = \text{[TODO: value]}$.

Alignment-based precision

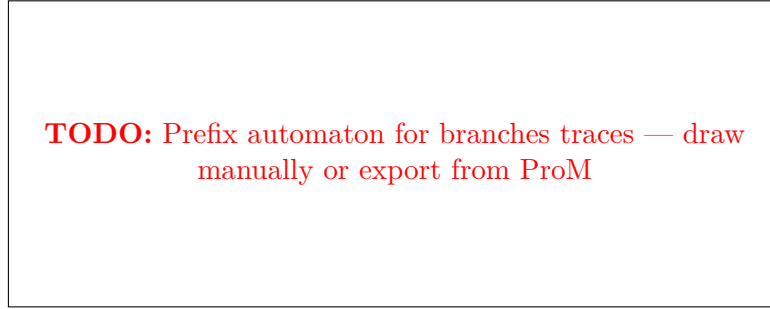


Figure 5: Prefix automaton of branches traces against N_1 branches subnet.

Branches subnet — prefix automaton.



Figure 6: Prefix automaton of Cameron's trace against N_1 users subnet.

Users subnet — prefix automaton.

Aggregated precision. The average alignment-based precision over both subnets is $\bar{p} = \frac{p_{\text{branches}} + p_{\text{users}}}{2} = [\text{TODO: value}]$.

Fitness and precision summary

Table 7: Aggregated conformance results for L_1^{Cameron} vs. N_1 .

Subnet	Token-replay fitness	Alignment precision
branches	[TODO: value]	[TODO: value]
users	[TODO: value]	[TODO: value]
Average	[TODO: value]	[TODO: value]

Checking C1 and C2 for Cameron

C1 (pull before push). [TODO: 2–3 sentences: does the fitness score reflect whether Cameron violated C1? For example: a fitness of 1.0 means every trace replays perfectly, implying Cameron always pulled before pushing (as required by C1). Or: explain what a fitness < 1 would mean in terms of C1.]

C2 (no branch-switching after commit). [TODO: 2–3 sentences: can the conformance results confirm or deny C2 for Cameron? Note that C2 is a cross-branch constraint whereas N1 is per-object — discuss whether flattening-based conformance is sufficient to detect C2 violations or whether it requires a different approach.]